Solution: Left and Right Sum Differences (easy)

Problem Statement

Given an input array of integers `nums`, find an integer array, let's call it `differenceArray,` of the same length as an input integer array.

Each element of `differenceArray`, i.e., `differenceArray[i],` should be calculated as follows: take the sum of all elements to the left of index `i` in array `nums` (denoted as `leftSum[i]`), and subtract it from the sum of all elements to the right of index `i` in array `nums` (denoted as `rightSum[i]`), taking the absolute value of the result. Formally:

differenceArray[i] = | leftSum[i] - rightSum[i] |

If there are no elements to the left/right of `i`, the corresponding sum should be taken as 0.

Examples

Example 1:

- **Input**: `[2, 5, 1, 6]`
- **Expected Output**: `[12, 5, 1, 8]`
- **Explanation**:
  - For i=0: |(0) - (5+1+6)| = |0 - 12| = 12
  - For i=1: |(2) - (1+6)| = |2 - 7| = 5
  - For i=2: |(2+5) - (6)| = |7 - 6| = 1
  - For i=3: |(2+5+1) - (0)| = |8 - 0| = 8

Example 2:

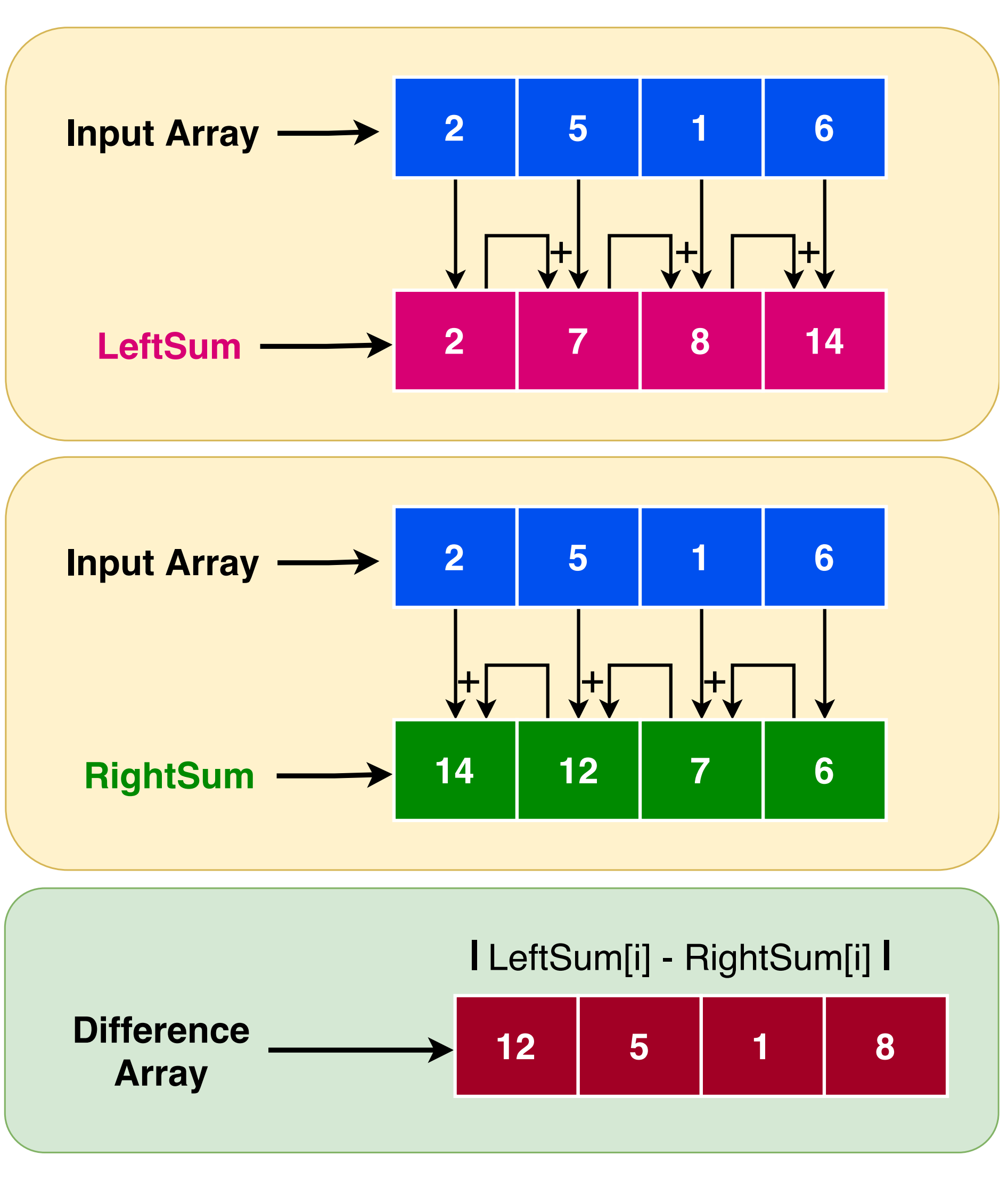
- **Input**: `[3, 3, 3]`
- **Expected Output**: `[6, 0, 6]`
- **Explanation**:
  - For i=0: |(0) - (3+3)| = 6
  - For i=1: |(3) - (3)| = 0
  - For i=2: |(3+3) - (0)| = 6

Example 3:

- **Input**: `[1, 2, 3, 4, 5]`
- **Expected Output**: `[14, 11, 6, 1, 10]`
- **Explanation**:
  - Calculations for each index i will follow the above-mentioned logic.

Solution

The algorithm takes a numerical approach to find the absolute differences between the sum of numbers to the left and to the right of each index in the input array, `nums`. It efficiently calculates two new arrays: `leftSum` and `rightSum` that respectively store the cumulative sums of elements to the left and right of every index `i` (inclusive). Afterward, it calculates the absolute difference between corresponding values in `leftSum` and `rightSum` to form the resulting array, `differenceArray`. This algorithm ensures minimized repeated calculations, as each sum is calculated in a linear pass and reused while determining the absolute differences.



1. **Initialize Arrays**:
   - A new array, `leftSum`, is initialized with the same length as `nums` to keep track of the cumulative sum to the left of each index.
   - Similarly, `rightSum` is initialized to keep track of the cumulative sum to the right of each index.
   - `differenceArray` is initialized to store the final results.
2. **Populate the leftSum Array**:
   - Traverse `nums` from left to right.
   - For each index `i`, `leftSum[i]` is determined by adding `nums[i]` to `leftSum[i-1]`. If `i` is 0, `leftSum[i]` is simply `nums[i]` since there are no elements to the left.
3. **Populate the rightSum Array**:
   - Traverse `nums` from right to left.
   - For each index `i`, `rightSum[i]` is determined by adding `nums[i]` to `rightSum[i+1]`. If `i` is the last index, `rightSum[i]` is `nums[i]` since there are no elements to the right.
4. **Calculate the differenceArray**:
   - Traverse from index 0 to length-1 of `nums`.
   - For each `i`, `differenceArray[i]` is calculated as the absolute difference between `leftSum[i]` and `rightSum[i]`.
5. **Return the Result**:
   - `differenceArray` is returned as it contains the final result.

Algorithm Walkthrough

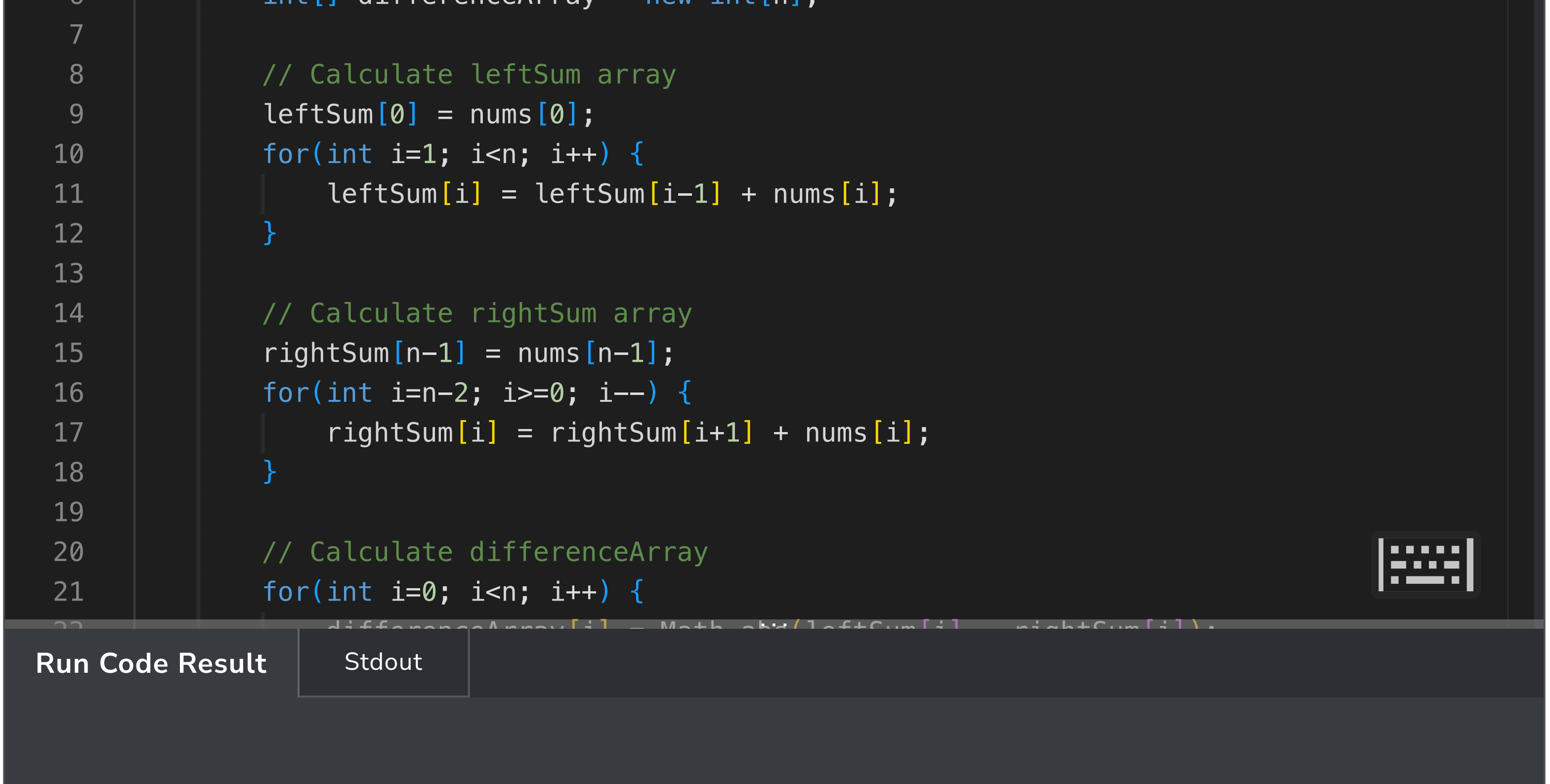
Consider the example where `nums = [2, 5, 1, 6]` for a detailed walkthrough.

- **Step 1**: **Initialize Arrays**:
  - `leftSum = [0, 0, 0, 0]`
  - `rightSum = [0, 0, 0, 0]`
  - `differenceArray = [0, 0, 0, 0]`
- **Step 2**: **Populate the leftSum Array**:
  - `i=0`: `leftSum[0] = nums[0] = 2`
  - `i=1`: `leftSum[1] = nums[1] + leftSum[0] = 5 + 2 = 7`
  - `i=2`: `leftSum[2] = nums[2] + leftSum[1] = 1 + 7 = 8`
  - `i=3`: `leftSum[3] = nums[3] + leftSum[2] = 6 + 8 = 14`
- **Step 3**: **Populate the rightSum Array**:
  - `i=3`: `rightSum[3] = nums[3] = 6`
  - `i=2`: `rightSum[2] = nums[2] + rightSum[3] = 1 + 6 = 7`
  - `i=1`: `rightSum[1] = nums[1] + rightSum[2] = 5 + 7 = 12`
  - `i=0`: `rightSum[0] = nums[0] + rightSum[1] = 2 + 12 = 14`
- **Step 4**: **Calculate the differenceArray**:
  - `differenceArray[i] = |leftSum[i] - rightSum[i]|` for each `i`
  - `differenceArray[0] = |2 - 14| = 12`
  - `differenceArray[1] = |7 - 12| = 5`
  - `differenceArray[2] = |8 - 7| = 1`
  - `differenceArray[3] = |14 - 6| = 8`
- **Step 5**: **Return the Result**:
  - The final array `differenceArray = [12, 5, 1, 8]` is returned as the output.

The remaining tasks include code development, complexity analysis, and test case generation. Should I proceed with these tasks sequentially or would you like to focus on a specific aspect first?

Code

Here is the code for this algorithm:



```java
public class Solution {
    public int[] findDifferenceArray(int[] nums) {
        int n = nums.length;
        int[] leftSum = new int[n];
        int[] rightSum = new int[n];
        int[] differenceArray = new int[n];

        // Calculate leftSum array
        leftSum[0] = nums[0];
        for(int i=1; i<n; i++) {
            leftSum[i] = leftSum[i-1] + nums[i];
        }

        // Calculate rightSum array
        rightSum[n-1] = nums[n-1];
        for(int i=n-2; i>=0; i--) {
            rightSum[i] = rightSum[i+1] + nums[i];
        }

        // Calculate differenceArray
        for(int i=0; i<n; i++) {
```

You must run your code first

Time Complexity

1. **Calculating Prefix and Suffix Sums**:
   - **Prefix Sum Calculation**: The loop that calculates the prefix sum runs `n` times, where `n` is the number of elements in the input array. Therefore, it has a time complexity of (O(n)).
   - **Suffix Sum Calculation**: Similarly, the loop for calculating the suffix sum also runs `n` times, resulting in a time complexity of (O(n)).
2. **Calculating Answer Array**:
   - The loop that calculates the absolute difference between prefix and suffix sums at each index also runs `n` times, contributing (O(n)) to the time complexity.

   Combining all these, the overall time complexity of the algorithm is (O(n) + O(n) + O(n) = O(3n)), which simplifies to (O(n)) because we typically ignore constant factors in Big O notation.

Space Complexity

1. **Prefix and Suffix Sum Arrays**:
   - The algorithm utilizes two additional arrays: `prefixSum` and `suffixSum`, each of size `n`. This contributes (2n) to the space complexity, i.e., (O(2n)).
2. **Answer Array**:
   - Additionally, an `answer` array of size `n` is used to store the final results. This adds an additional (O(n)) to the space complexity.

   Combining these, the overall space complexity is (O(2n) + O(n) = O(3n)), which simplifies to (O(n)) when considering Big O notation.

← Previous
Problem 3: Left And Right Sum Differences (Easy)

Next →
Problem 4: Find The Highest Altitude(Easy)

☐ Mark as Completed